

CloudContextGuard

Context-Aware Cloud Access Control Using IAM and Trusted VPC
Boundaries

SECURITY AND PRIVACY IN CLOUD

**Submitted by: PRIYAANSH JAISWAL & SHASHANK SUNIL
HEBBAL**

University: RV UNIVERSITY

Department: SOCSE

Academic Year: 2026-2027

Repository: <https://github.com/priyaansh21/CloudContextGuard>

Abstract

Cloud storage breaches are frequently not the result of a broken permission model, but of a permission model that is trusted too much: a single valid Identity and Access Management (IAM) permission is often treated as sufficient proof of authorization, irrespective of network origin, multi-factor authentication (MFA) status, or contextual risk. CloudContextGuard is a fully implemented, locally-run simulation of a context-aware cloud authorization system that challenges this assumption. The system combines five independent, deterministic, rule-based evaluation engines: an IAM engine, a Virtual Private Cloud (VPC) trust-boundary engine, a resource-policy engine, an MFA evaluator, and a risk-scoring engine into a single authoritative decision engine that alone produces the final ALLOW or DENY outcome. Every decision is enforced strictly server-side: a client cannot submit, influence, or override the risk score or the final decision. The system is built on a FastAPI backend, a SQLite database that is the single source of truth for identities, resources, network boundaries, policies, and the full audit trail, and a React dashboard that visualizes and drives the system, including a live policy editor and a six-scenario attack simulation laboratory. The project's central, empirically reproduced finding is that a genuinely valid IAM permission held by a real, seeded identity is still denied when the request originates from an untrusted network without MFA, demonstrating that IAM permission is necessary but not sufficient for authorization. The system further demonstrates that authorization policy is administrator-configurable at runtime, directly from the database, and takes effect on the very next request with no code change or restart. The implementation is validated by 82 automated backend tests (82 passed, 0 failed), a 17-point portability check (17 of 17 passed), and a successful frontend production build, establishing CloudContextGuard as a reproducible, evidence-based demonstration of contextual, zero-trust cloud authorization.

Table of Contents

Abstract	1
Table of Contents	2
1. Introduction	3
2. Problem Statement	3
3. Objectives	4
4. Existing System and Its Limitations.....	4
5. Proposed System	5
6. System Architecture	5
7. Technology Stack	7
8. System Workflow	7
9. Authorization Decision Model	9
10. Risk Engine.....	10
11. Dynamic Policy Management	12
12. Security Controls	13
13. Attack and Security Scenarios	13
14. Main Topic 19 Demonstration — Valid IAM Permission, Denied by Context.....	15
15. Dynamic Policy Demonstration	18
16. Frontend and Dashboard.....	20
17. API Overview	24
18. Database Design	24
19. Security Events and Alerts	26
20. Testing and Validation	27
21. Results	28
22. Limitations	28
23. Future Work	29
24. Conclusion	29
25. References	30

1. Introduction

CloudContextGuard is a locally-run, fully working simulation of a context-aware cloud authorization system. It is built to demonstrate, with a real running application rather than a slide deck or a design diagram, that a cloud access-control decision should never rest on identity permission alone. Every request submitted to the system is evaluated against identity (IAM), network origin (VPC trust boundary), multi-factor authentication, a per-resource policy, and a deterministic risk score, and only after all five evaluations complete is a final ALLOW or DENY decision produced.

The system consists of three cooperating parts: a FastAPI backend that is the sole authorization authority, a SQLite database that is the single source of truth for identities, resources, network boundaries, policies, and the complete audit history, and a React dashboard that lets an operator observe and drive the system, including a live policy editor and an attack-simulation laboratory. No real AWS account, IAM credential, VPC, or cloud resource is used or contacted anywhere in this project every “resource,” “VPC,” and “identity” referenced in this report is a row in a local SQLite database created specifically for this simulation.

Cloud storage breaches are frequently not the result of a broken permission model, but of a permission model that is trusted too much: a single leaked credential, used from anywhere, is treated as sufficient proof of authorization. CloudContextGuard implements and demonstrates an alternative in which IAM permission is necessary but never sufficient, and every access decision is the product of several independent, explainable checks working together.

2. Problem Statement

Given a valid IAM permission, most conventional authorization systems grant access unconditionally. This project asks a more specific question: what additional, independently enforced controls are needed so that a valid IAM permission, used from an untrusted network or without multi-factor authentication, is still denied and how can those controls be made administrator-configurable at runtime without weakening the guarantee that the server, and never the client, is authoritative over the decision?

Stolen-credential and unexpected-network access are among the most common real-world cloud storage attack patterns. An attacker who obtains a leaked API key or session token inherits every permission that identity holds, from any network, with no additional friction, under a conventional IAM-only model. A system that can concretely show “this identity had a genuine, valid permission, and was still denied because of where the request came from” is a stronger teaching and research artifact than a diagram, because every claim it makes can be independently re-run and re-verified against a live API.

3. Objectives

To design, implement, and empirically verify a rule-based, explainable, dynamically-configurable authorization model satisfying the following objectives:

1. Authorization decisions are enforced server-side; a client cannot submit or influence the decision, risk score, or any other server-controlled field.
2. IAM permission alone does not guarantee access.
3. Network context (VPC trust boundary) can independently deny an otherwise-permitted request.
4. MFA requirements are enforced independently of IAM and network context.
5. A deterministic, explainable risk score can independently deny a request above a documented threshold.
6. Resource policy enabled state, required role, trusted VPC, MFA, external access, and maximum risk is enforced on every request.
7. Policy changes made by an administrator at runtime affect the very next authorization decision, with no code or deployment change.
8. Every decision is captured in an accurate, tamper-resistant audit trail.

4. Existing System and Its Limitations

Conventional access-control demonstrations and many production systems fall into one of three categories, each of which CloudContextGuard was designed to address:

- A policy model described only in the abstract diagrams and documents with no running implementation to independently verify.
- IAM-only permission checks with no contextual (network, MFA, risk) enforcement a valid permission is treated as sufficient on its own, regardless of where or how the request was made.
- Policy hardcoded in source code, such that any policy change requires a code change and a redeploy, making the system slow to respond to a changing threat posture.

CloudContextGuard addresses all three limitations directly: it is a running, independently testable system rather than a design artifact; it enforces network, MFA, and risk context independently of IAM, so a valid permission is never treated as sufficient by itself; and its policy is live, database-backed, and administrator-editable at runtime, so a policy change takes effect on the very next request with no code change or restart.

5. Proposed System

CloudContextGuard is a FastAPI service exposing POST /api/access/request as the single authorization entry point, backed by five pure-function “engines” (IAM, VPC, Policy, MFA, and Risk) that are orchestrated by an access service and combined by a single central decision engine. A SQLite database holds both the reference catalog (users, roles, permissions, resources, VPCs, policies) and the full audit trail (access requests, security events, alerts). A React frontend is a genuine client of this API it never computes a security decision itself; every decision, risk score, and pass/fail value shown in the dashboard is read directly from a live API response.

The scope of the implemented system is summarized below.

In Scope

- A simulated IAM / VPC / MFA / risk / policy authorization pipeline.
- A SQLite-backed catalog of synthetic users, roles, resources, and network boundaries.
- A REST API exposing every operation used by the frontend.
- A React administrative and demonstration frontend, including a live policy editor.
- An attack simulation laboratory exercising the real authorization pipeline.
- An automated test suite validating all of the above.

Out of Scope

- Any connection to a real cloud provider (AWS, Azure, or otherwise).
- Any real credential or identity provider.
- Real network traffic or packet capture.
- Machine-learning-based risk scoring.
- Production-grade operational concerns such as rate limiting, horizontal scaling, or a distributed database.

Note: Items explicitly out of scope are documented as limitations (Section 22) and future work (Section 23), and are never implemented or claimed as working features of the current system.

6. System Architecture

The system follows a strict layered architecture in which every layer has a single, well-defined responsibility. The React frontend never computes an authorization outcome; it only renders what the FastAPI API returns. The FastAPI application routes each request to the access service, which resolves the relevant database rows and calls each of the five engines in turn. Each engine is a pure function: it accepts already-resolved, explicit inputs and returns a structured, explainable result object never a bare boolean. No engine other than the decision engine is permitted to produce a final ALLOW/DENY outcome, and no engine queries the database directly; all resolution and orchestration is centralized in the access service. Figure 1 shows the complete request path from the frontend through to persistence and the resulting dashboard aggregation.

CloudContextGuard — System Architecture

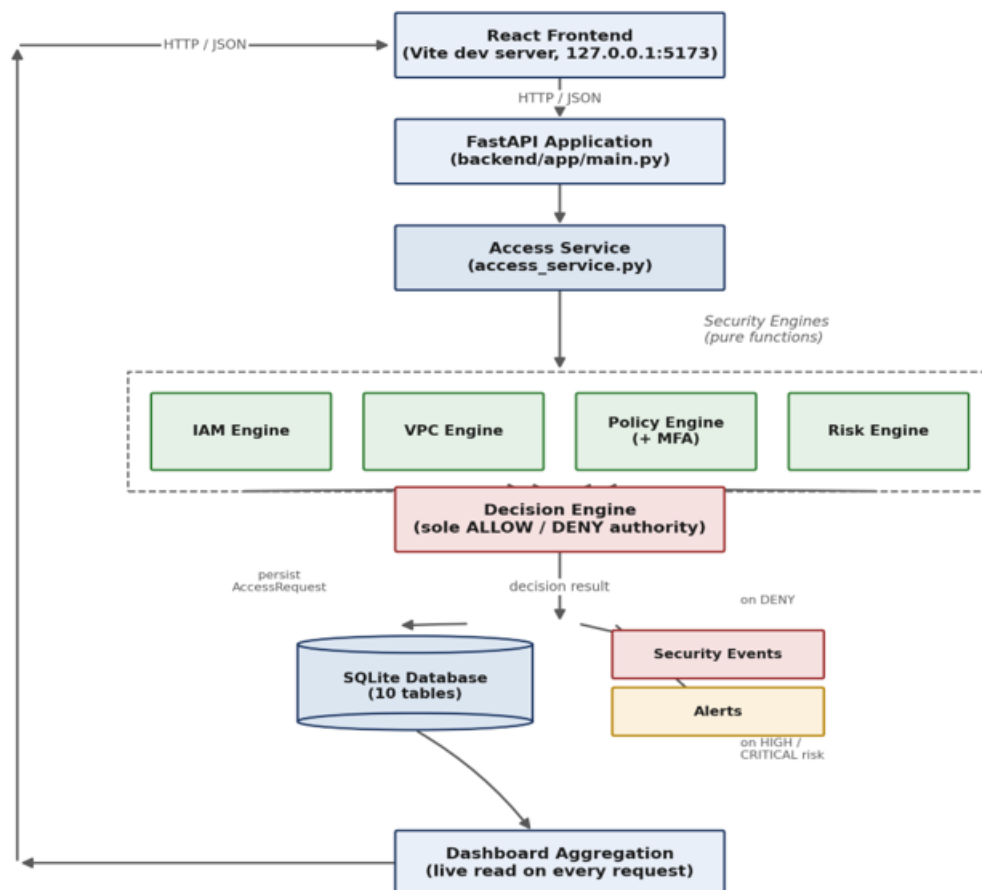


Figure 1. CloudContextGuard system architecture — request path from the React frontend through the FastAPI access service, the five security engines, the decision engine, and persistence.

Component Description

Table 1. Core architectural components and their responsibilities.

Component	Location	Responsibility
API layer	backend/app/api/*.py	HTTP routing and request/response schema validation; no business logic.
Access service	backend/app/services/access_service.py	Orchestrates one access request through every engine and persists the outcome.
Policy service	backend/app/services/policy_service.py	Validates and applies administrator policy edits; writes the audit trail.
Seed service	backend/app/services/seed_service.py	Idempotent baseline catalog and demonstration-identity seeding.
IAM engine	backend/app/engine/iam_engine.py	Does the identity's role hold a permission matching this action/resource?
VPC engine	backend/app/engine/vpc_engine.py	Did the request originate from the resource's currently trusted network boundary?
Policy engine (+ MFA)	backend/app/engine/policy_engine.py	Is the resource's policy enabled, is the

Component	Location	Responsibility
		required role satisfied, and was MFA satisfied?
Risk engine	backend/app/engine/risk_engine.py	Deterministic 0–100 risk score built from named, explainable factors.
Decision engine	backend/app/engine/decision_engine.py	Combines every result above into the final ALLOW/DENY — the sole authority for the decision.
Database layer	backend/app/db/*.py	SQLAlchemy 2.0 models and schema-version safety mechanism.
Frontend pages	frontend/src/pages/*.jsx	Dashboard, Access Requests, IAM, VPC Security, Storage, Security Events, Alerts, Policies, Attack Simulator.

7. Technology Stack

The technologies listed below are the actual, verified dependencies of the implementation; no cloud platform, managed service, or external API is used anywhere in the project.

Table 2. Technology stack, by layer.

Layer	Technologies
Backend	Python, FastAPI 0.115, SQLAlchemy 2.0, Pydantic 2.10, pydantic-settings, Uvicorn, pytest, httpx (via FastAPI's TestClient)
Frontend	React 19, Vite 8, React Router 7, Tailwind CSS 4, Recharts, Lucide React icons
Database	SQLite (file-based, portable; part of the project's own data/ directory, resolved dynamically at runtime)
Testing / QA	pytest (82 backend tests), tests/verify_portability.py (17 portability checks), npm run build (frontend production build)

8. System Workflow

Every access request follows the same fixed sequence of steps, regardless of the outcome. The user and resource named in the request are first resolved from the database (a 404 is returned if either does not exist); the IAM, VPC, resource-policy, and MFA checks are then evaluated in turn; a repeated-failure lookup scans the requesting user's denied requests in the preceding 15 minutes; the risk score is calculated from the accumulated factors; and the decision engine combines every result into a final ALLOW or DENY. The outcome is always persisted as an AccessRequest row. A DENY always creates a SecurityEvent; a HIGH or CRITICAL risk level always creates an Alert, regardless of whether the request was ultimately allowed or denied. Figure 2 shows this complete sequential workflow, including both branch points.

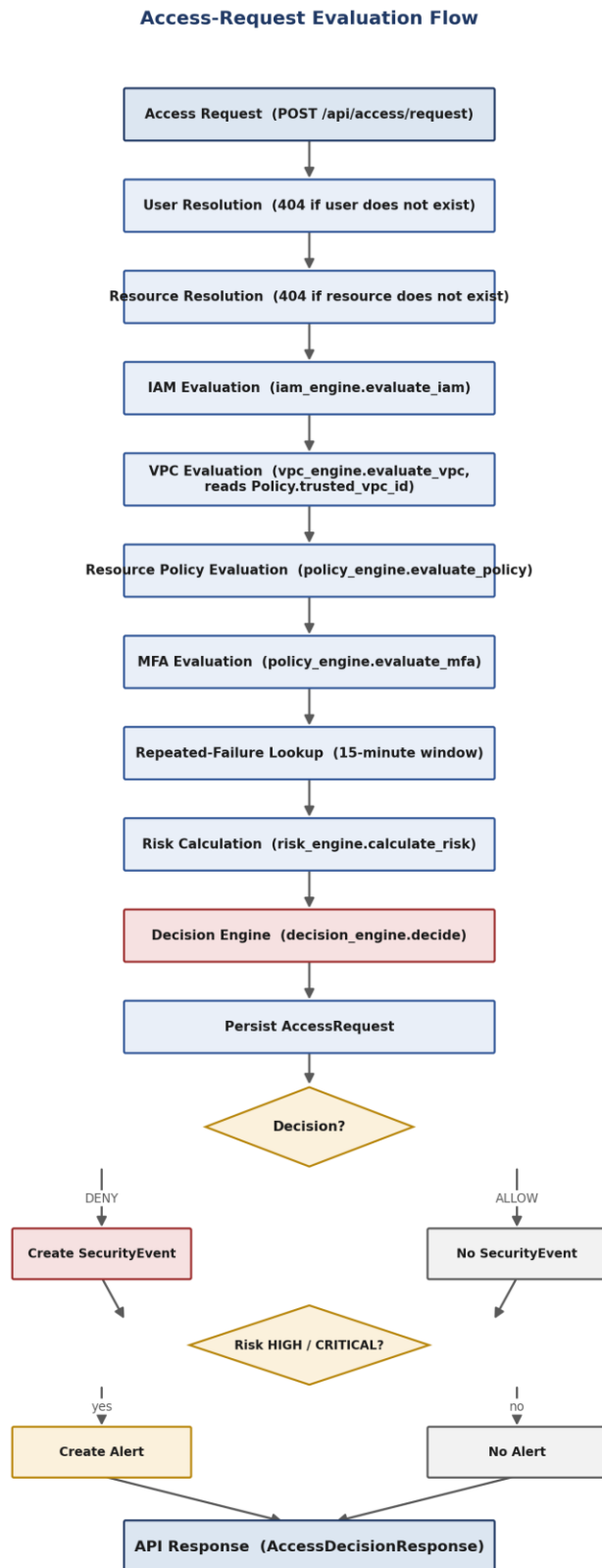


Figure 2. End-to-end authorization workflow, from the incoming access request to the API response, including the SecurityEvent and Alert branch points.

9. Authorization Decision Model

The decision engine (backend/app/engine/decision_engine.py) is the only component in the system permitted to produce a final ALLOW or DENY outcome. It evaluates a fixed priority order of conditions; the first condition that applies determines the reason a request is denied, though every other check still runs and is reported in full, regardless of which check ultimately caused the denial. This is the central research demonstration of the project: a genuinely valid IAM permission (priority step 1 passing) is not, by itself, sufficient for access, because steps 2 through 8 are independent, additional gates. Figure 3 illustrates this exact priority order as implemented in the source code.

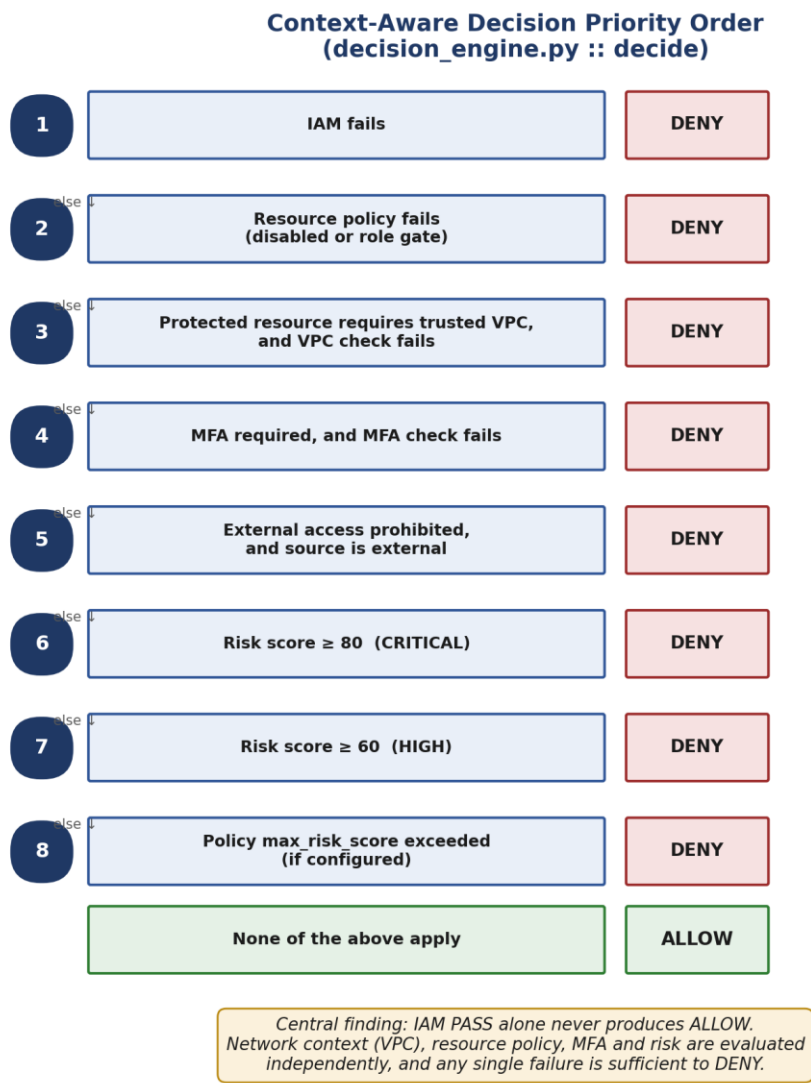


Figure 3. Decision engine priority order — IAM permission alone (step 1) is necessary but not sufficient; steps 2–8 are independent, additional gates before ALLOW is reached.

Table 3. Decision engine priority order (decision_engine.py::decide).

Step	Condition	Outcome
1	IAM fails	DENY immediately (all other checks still run and are reported)
2	Resource policy fails (disabled, or required role not satisfied)	Contributes to DENY
3	Protected resource requires a trusted VPC and the VPC check failed	Contributes to DENY
4	MFA is required and failed	Contributes to DENY
5	External access is not permitted and the source is external	Contributes to DENY
6	Risk score ≥ 80	Contributes to DENY (reason names it CRITICAL)
7	Risk score ≥ 60	Contributes to DENY (reason names it HIGH)
8	Policy-specific max_risk_score configured and exceeded (checked only if 6–7 did not already deny)	Contributes to DENY, as an additional safeguard
9	None of the above apply	ALLOW

If one or more of steps 2 through 8 apply, the final decision is DENY, and the response reason string is composed from every failing check's own reason for example, “IAM authorization succeeded, but the request originated outside the trusted VPC and the protected resource requires MFA and the request was assessed as CRITICAL risk (score 100).” This is never a fixed template disconnected from what actually happened; it is assembled directly from the real results of each check on each request.

10. Risk Engine

The risk engine (backend/app/engine/risk_engine.py) produces a transparent, deterministic, rule-based score between 0 and 100 no machine learning and no hidden weighting is used anywhere in this project. Each risk factor below is independent and additive; a single request can accumulate any combination of these factors.

Table 4. Risk factors and their point values, as implemented in risk_engine.py.

Factor	Points	Trigger Condition
External network	+40	The VPC check failed for this request.
CONFIDENTIAL resource	+30	The resource's classification is CONFIDENTIAL.
RESTRICTED resource	+40	The resource's classification is RESTRICTED (mutually exclusive with CONFIDENTIAL).
Missing MFA	+30	MFA is required for this resource and was not provided.
Unknown / untrusted source	+20	The source network is not a known, trusted VPC.
Repeated failed requests	+10	Two or more denied requests by the same user in the last 15 minutes.
Suspicious request	+20	request_type == "SUSPICIOUS".

The raw score is the sum of every triggered factor's points, then capped so that it never exceeds 100:

$$\text{score} = \min(\text{sum}(\text{triggered factor points}), 100)$$

Table 5. Risk level thresholds (risk_engine.py, evaluated top-down).

Score Range	Risk Level
0–29	LOW
30–59	MEDIUM
60–79	HIGH
80–100	CRITICAL

Worked Example

Request: developer01, GetObject, financial-records, source VPC external, MFA not provided, request_type = "SUSPICIOUS".

Table 6. Worked risk-scoring example (source: docs/research/SECURITY_DECISION_MODEL.md).

Factor	Triggered?	Points
External network	Yes — VPC check fails (financial-records requires vpc-finance)	40
CONFIDENTIAL resource	Yes — financial-records is CONFIDENTIAL	30
Missing MFA	Yes — required, not provided	30
Unknown / untrusted source	Yes — external is not a trusted VPC	20
Suspicious request	Yes	20
Raw total		140
Capped score		100 (CRITICAL)

In this example, developer01 also has no IAM permission on financial-records, so IAM already fails independently at priority step 1 the final decision is DENY regardless of the risk score. The project's central demonstration of risk and network context independently reinforcing a contextual denial, where IAM does pass, is presented in Section 14.

11. Dynamic Policy Management

Policy rows are the authoritative, administrator-editable configuration for each resource: required role, trusted VPC, MFA requirement, maximum risk score, external-access setting, and enabled state. The access service queries the Policy table fresh on every single access evaluation there is no cache anywhere in the request path. PUT /api/policies/{id} is the only way to change a policy; every change is validated, applied, and recorded as a POLICY_CHANGED security event describing exactly which field changed, from what value, to what value. Figure 4 illustrates the complete, real, executed experiment that proves this mechanism: a policy edit made through the API takes effect on the very next access evaluation, with no application restart and no code change.

Dynamic Policy Evaluation — 6-Step Live Experiment

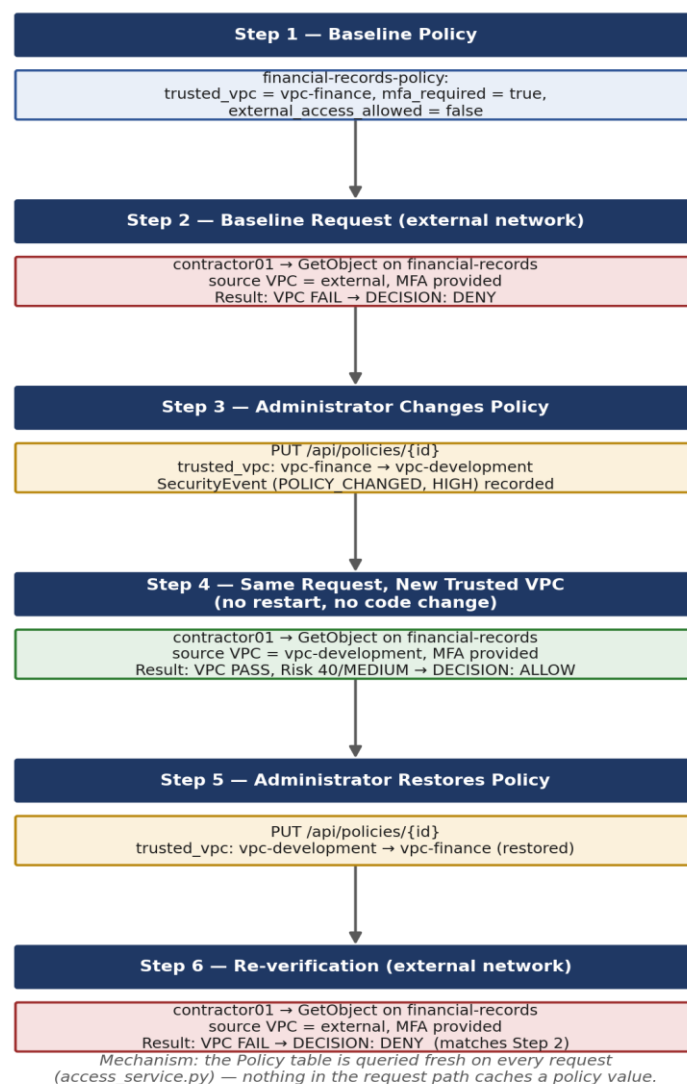


Figure 4. Dynamic policy evaluation experiment — baseline denial, a live policy edit, the resulting allow, and restoration to the original, secure configuration.

This mechanism is what allows CloudContextGuard's authorization policy to respond to a changing security posture without a redeploy the exact limitation identified in conventional, hardcoded-policy systems (Section 4). The full worked experiment, with real, API-verified results at every step, is presented in Section 15.

12. Security Controls

In addition to the five-engine authorization model itself, a dedicated hardening and validation pass independently verified the following controls, each covered by automated tests in `backend/tests/test_security_validation.py`:

- Server-side authority: a client-supplied “decision” field in an access request cannot force an outcome; the backend always recomputes the decision from the database.
- Mass-assignment resistance: extra fields submitted in an access request or a policy update (for example `role`, `is_admin`, `risk_score`, `severity`) are ignored, never applied.
- Injection resistance: SQL-like, script-like, and path-traversal-like input strings are treated as inert data; no raw SQL interpolation exists anywhere in the codebase.
- Referential integrity: foreign-key constraints are enforced on every connection (`PRAGMA foreign_keys=ON`), with zero orphaned rows verified by the test suite.
- Safe error responses: 404 and 422 error responses contain no stack traces, file paths, or environment data.
- CORS allowlist: cross-origin requests are restricted to a specific, explicit origin allowlist never a wildcard combined with credentials.
- Security response headers: `X-Content-Type-Options`, `X-Frame-Options`, and `Referrer-Policy` are attached to every response by a lightweight middleware.
- Schema-version safety: a lightweight schema-version mechanism guarantees that normal startup never silently drops a table or deletes data, and that an incompatible on-disk schema fails startup loudly instead of being silently recreated.

13. Attack and Security Scenarios

Seven threat scenarios (T1–T7) were designed and exercised entirely within CloudContextGuard's own local simulation: a synthetic user submits a request to `POST /api/access/request`, and the real IAM, VPC, MFA, policy, and risk engines evaluate it. No real network, cloud account, or credential is ever involved. Figure 5 shows the generalized request path shared by every threat scenario, from an external actor with a compromised or misused — but genuinely valid — credential, through context evaluation, to the decision engine and the resulting audit trail.

Threat Model — Generalized Request Path (T1-T5)

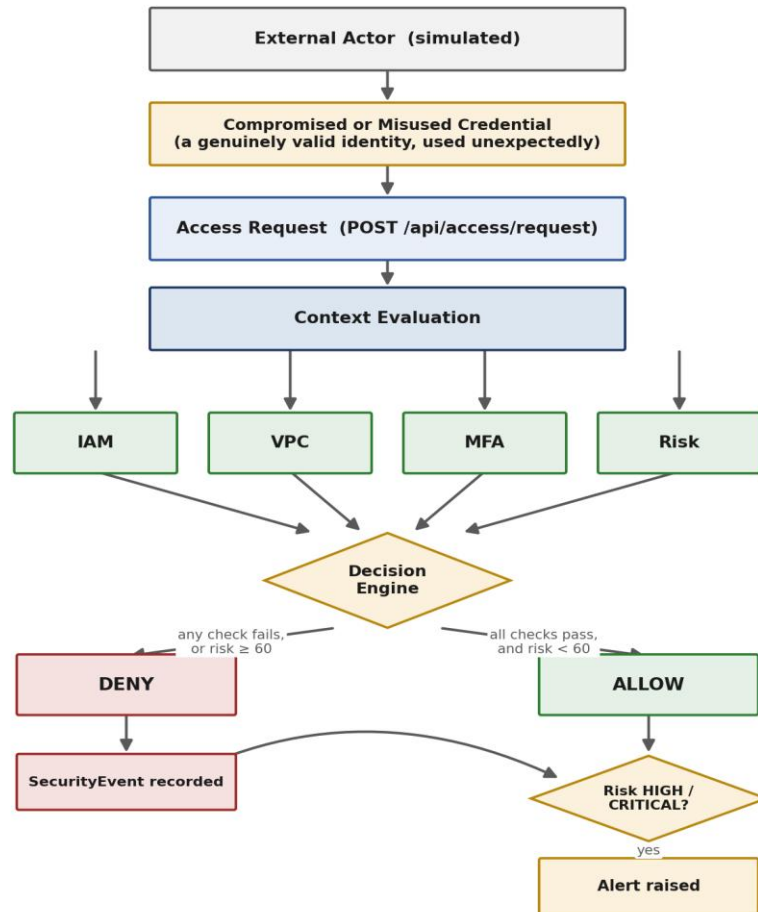


Figure 5. Generalized threat model request path (T1–T5) — every simulated threat is evaluated through the same context-evaluation and decision pipeline.

Table 7. Threat model summary (source: *THREAT_MODEL.md*).

ID	Threat	Attack Condition	Expected Result
T1	Compromised developer credentials	developer01 requests a permitted resource from an untrusted external network, no MFA	DENY (VPC + MFA fail)
T2	Access from outside the trusted VPC	contractor01 (real GetObject permission) requests financial-records from external	DENY (IAM passes, VPC fails)
T3	Missing MFA	finance01 requests financial-records from the correct trusted VPC, mfa=false	DENY (IAM + VPC pass, MFA fails)
T4	Unauthorized resource access	developer01 requests credentials-vault (no granted permission)	DENY (IAM fails)
T5	Privilege escalation attempt	developer01 requests DeleteObject on credentials-vault (RESTRICTED, requires AdminRole)	DENY (IAM + resource policy fail)
T6	Repeated unauthorized requests	developer01 submits 3+ sequential denied requests within 15 minutes	Escalating risk score; DENY throughout
T7	Sensitive storage exposure	A policy is misconfigured with external_access_allowed=true on a CONFIDENTIAL/RESTRICTED resource	Visible administrative warning; no fabricated denial

T1 through T5 are additionally exercised live through the Attack Simulator (frontend/src/pages/Simulator.jsx), which submits a genuine POST /api/access/request for each scenario and renders whatever the API actually returns the frontend never computes or fakes a decision, a risk score, or a pass/fail result.

14. Main Topic 19 Demonstration — Valid IAM Permission, Denied by Context

This is the project's central research demonstration. It shows, using a genuinely running system rather than a hypothetical, that a valid IAM permission is necessary but not sufficient for access. As a baseline, Figure 6 shows a normal, fully trusted request that is correctly ALLOWED: developer01 requests GetObject on project-data from vpc-development, with MFA provided IAM, VPC, resource policy, and MFA all pass, the risk is assessed as LOW, and the final decision is ALLOW.

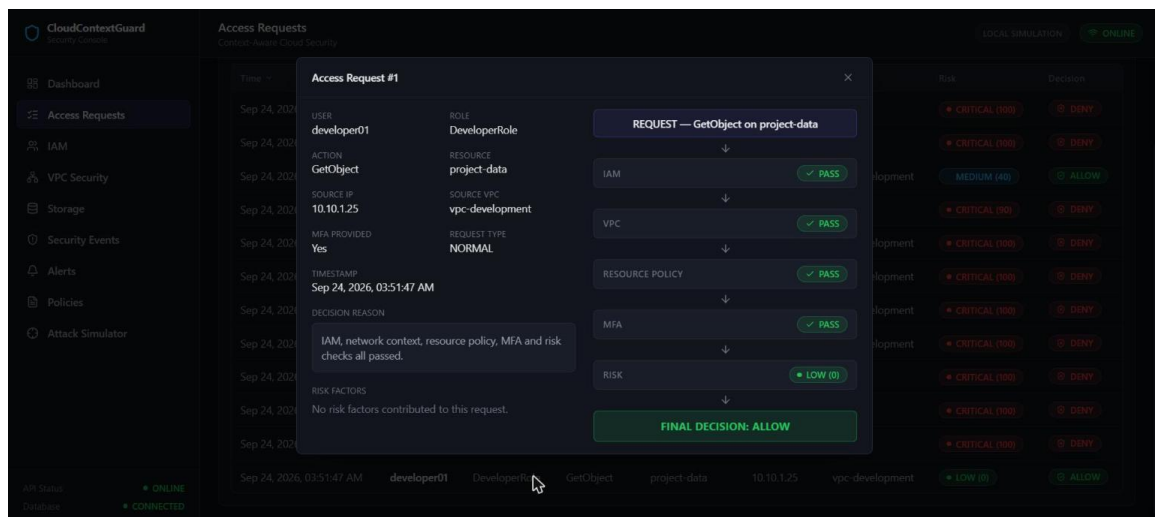


Figure 6. Baseline trusted request (developer01, GetObject, project-data, vpc-development, MFA provided)
— all checks pass, LOW risk, FINAL DECISION: ALLOW.

The central demonstration is the direct contrast to this baseline: contractor01 holds a genuine, real GetObject permission on financial-records — IAM passes. The same request, however, is issued from an external, untrusted network without MFA, with request_type="SUSPICIOUS". Figure 7 shows the real, captured result of this exact request against the running application.

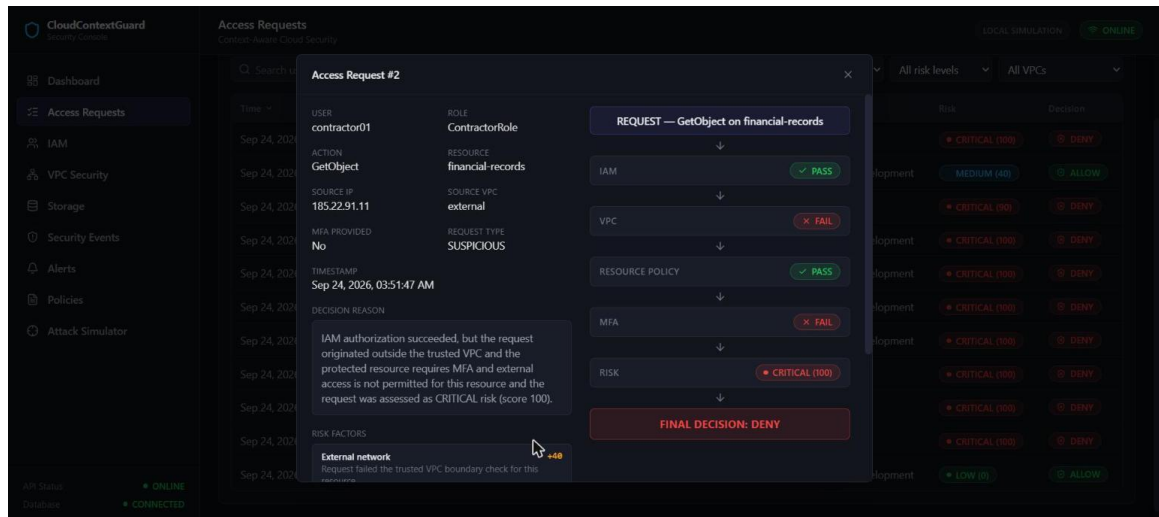


Figure 7. Central demonstration: contractor01, GetObject, financial-records, external network, MFA not provided — IAM PASS, VPC FAIL, MFA FAIL, CRITICAL risk (100), FINAL DECISION: DENY.

Table 8. Verified result of the primary contextual-denial demonstration (source: docs/research/RESULTS.md).

Check	Result
IAM	PASS
VPC	FAIL
MFA	FAIL
Risk	100 / CRITICAL
Final decision	DENY

A genuinely valid IAM permission (ContractorRole does hold GetObject:financial-records) was not sufficient to grant access once network context and MFA were independently evaluated. A SecurityEvent (event_type="BLOCKED_DATABASE_ACCESS", action_taken="BLOCKED") and an Alert (severity="CRITICAL", status="OPEN") were both created and independently confirmed via the API. The same scenario, re-executed through the Attack Simulator's "Trusted Identity, Untrusted Network" scenario, produces an identical result, shown in Figures 8 and 9: the simulator's own pass/fail badges and the full risk-factor breakdown, both read live from the same API response.

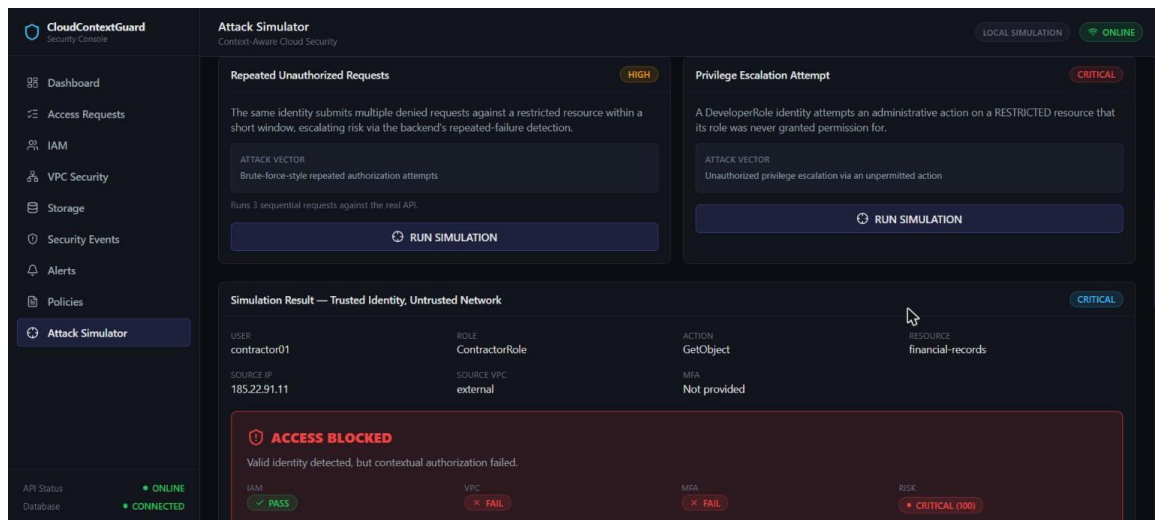


Figure 8. Attack Simulator — “Trusted Identity, Untrusted Network” scenario result: ACCESS BLOCKED, with IAM PASS / VPC FAIL / MFA FAIL / RISK CRITICAL(100) badges.

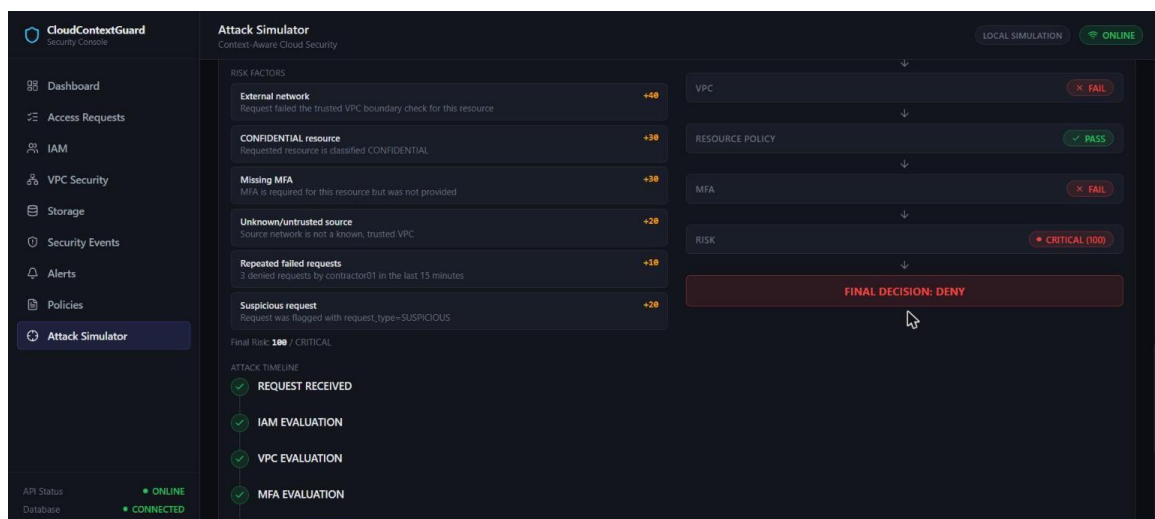


Figure 9. Full risk-factor breakdown for the same scenario — External network (+40), CONFIDENTIAL resource (+30), Missing MFA (+30), Unknown/untrusted source (+20), Repeated failed requests (+10), Suspicious request (+20), capped at 100/CRITICAL.

15. Dynamic Policy Demonstration

This demonstration proves that a live policy edit, made by an administrator through the Policy Management page, is read from the database and applied to the very next access evaluation with no source-code change and no application restart. The resource under test is financial-records (classification CONFIDENTIAL); the identity under test is contractor01, which holds a genuine GetObject permission on it.

Step-by-Step Experiment

1. Baseline policy: financial-records-policy has trusted_vpc = "vpc-finance", mfa_required = true, external_access_allowed = false, is_enabled = true.
2. Baseline request from an external network (MFA provided): IAM PASS, VPC FAIL (expected vpc-finance, actual external) DENY.
3. The administrator changes the policy's trusted_vpc from "vpc-finance" to "vpc-development" through PUT /api/policies/{id}. A SecurityEvent (event_type="POLICY_CHANGED", severity="HIGH") is recorded describing exactly what changed.
4. The identical request is resubmitted, now from vpc-development, with MFA provided shown in Figure 10.
5. Policy restored: the administrator reverts trusted_vpc back to "vpc-finance" through the same API.
6. Re-verification: the original external-network request is resubmitted and produces exactly the original result shown in Figure 11.

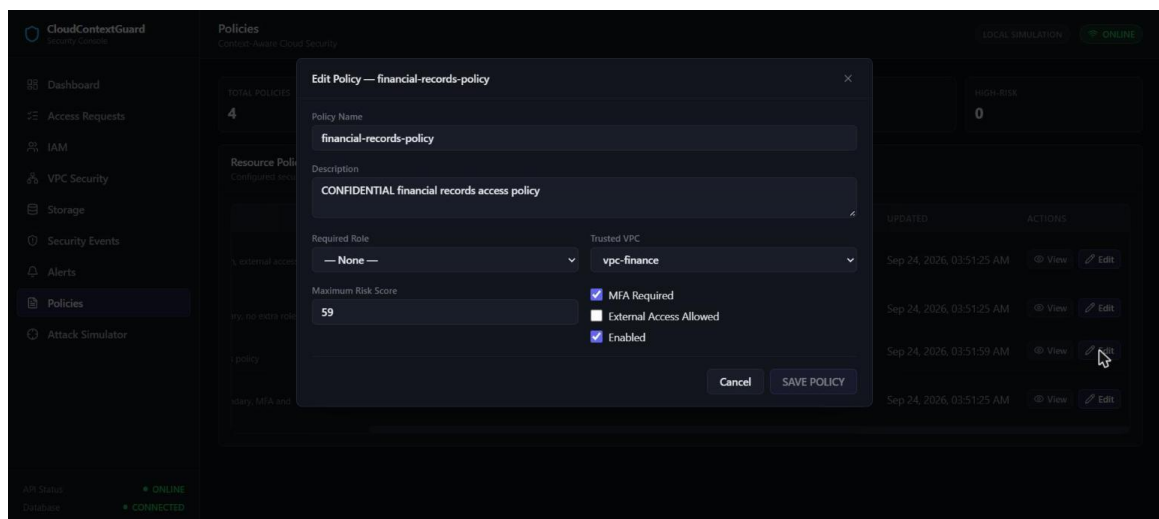


Figure 10. Policy Management — live editor for financial-records-policy, showing the Trusted VPC dropdown, MFA Required checkbox, and Max Risk Score field used to perform Step 3 of the experiment.

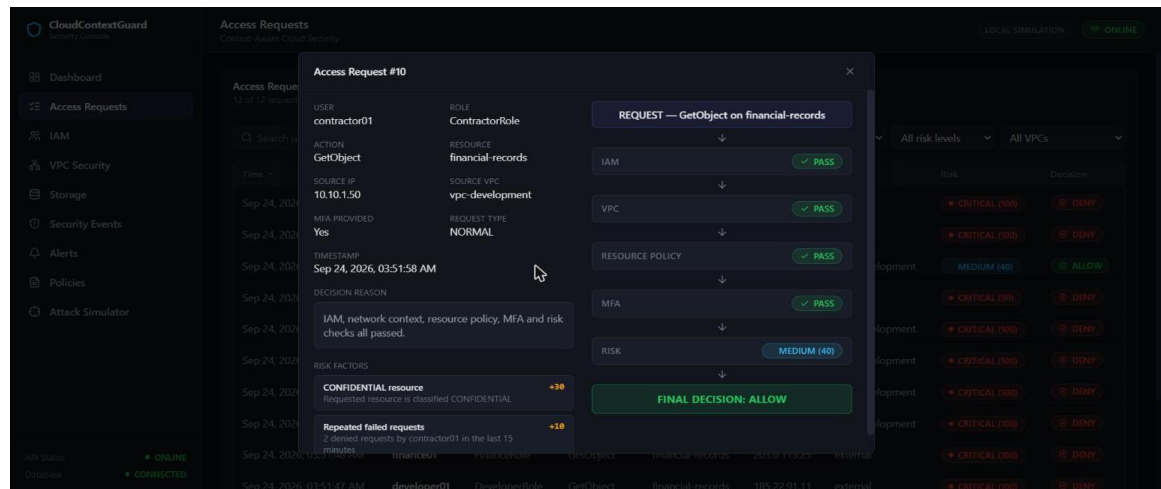


Figure 11. Step 4 result: contractor01, GetObject, financial-records, now from vpc-development with MFA — IAM PASS, VPC PASS, MEDIUM risk (40), FINAL DECISION: ALLOW.

Table 9. Verified result after the live policy change (source: docs/research/RESULTS.md).

Check	Result
IAM	PASS
VPC	PASS
Risk	40 / MEDIUM
Final decision	ALLOW (determined by the remaining policy/risk controls, not predetermined)

The VPC engine's expected_vpc field itself read “vpc-development” after the change — direct evidence that the evaluation used the freshly updated database value, not a stale or compiled-in one. With IAM and VPC both passing, the remaining policy and risk controls genuinely determined the ALLOW outcome; it was not a predetermined result. Figure 12 shows Step 6, in which the policy has been restored to its original value and the external-network request is denied again, exactly matching the original baseline of Step 2.

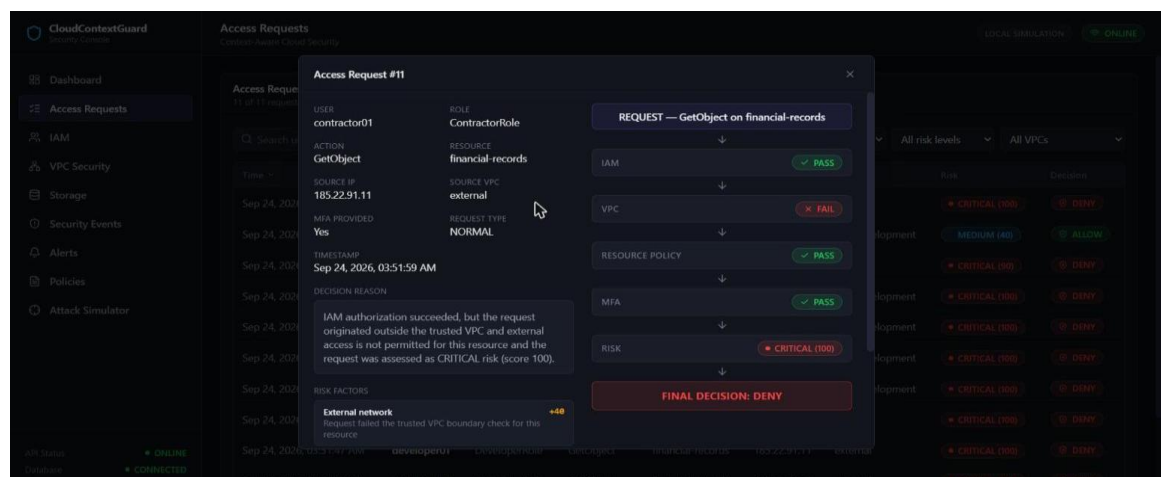


Figure 12. Step 6 re-verification: after the policy is restored, the identical external-network request is DENIED again, exactly matching the original baseline (VPC FAIL, CRITICAL risk).

This experiment directly demonstrates the required property: a policy change made through the API is read from the database and applied to the very next access evaluation, with no source-code change and no restart, and the system can be returned to its original, secure configuration by the same mechanism.

16. Frontend and Dashboard

The frontend is a React 19 single-page application built with Vite, styled with Tailwind CSS, routed with React Router, and charted with Recharts. It holds no authorization logic of its own: every decision, risk score, and pass/fail value displayed anywhere in the interface is read directly from an API response, through a single centralized API client (frontend/src/services/api.js) — no component builds a request URL or computes a security outcome directly.

Dashboard

The Dashboard page presents a live, database-derived summary — total requests, allowed/denied counts, high-risk requests, critical events, and open alerts — alongside decision and risk distribution charts, shown in Figure 13.

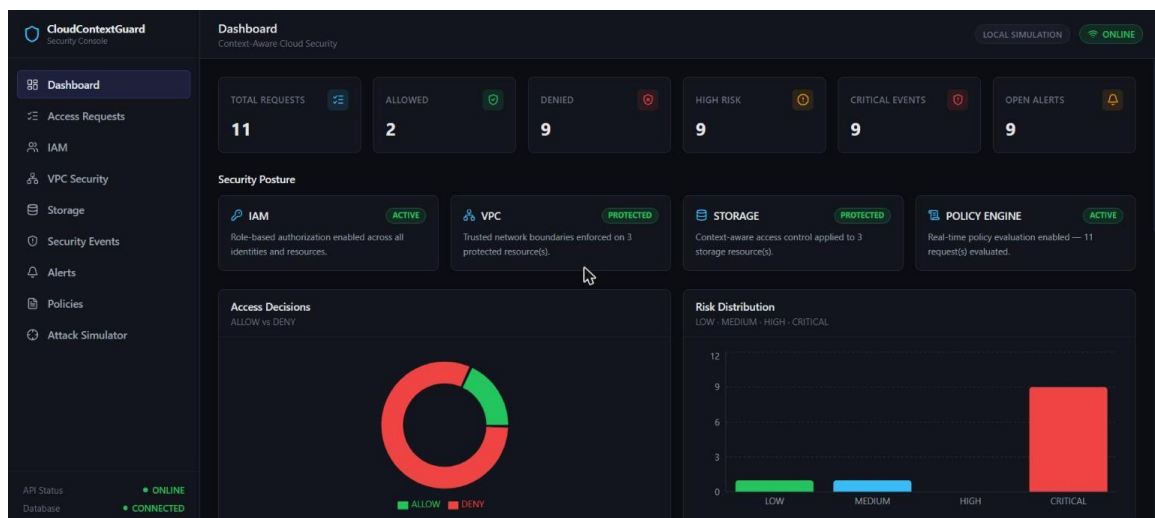


Figure 13. Dashboard overview — live-computed request totals, decision distribution, and risk distribution for the current demonstration dataset.

Access Requests

The Access Requests page is the full audit log of every evaluated request, with a detail view exposing the complete decision pipeline for any individual request, shown in Figure 14.

Time	User	Role	Action	Resource	Source IP	VPC	Risk	Decision
Sep 24, 2026, 03:51:59 AM	contractor01	ContractorRole	GetObject	financial-records	185.22.91.11	external	CRITICAL (100)	DENY
Sep 24, 2026, 03:51:58 AM	contractor01	ContractorRole	GetObject	financial-records	10.10.1.50	vpc-development	MEDIUM (40)	ALLOW
Sep 24, 2026, 03:51:58 AM	contractor01	ContractorRole	GetObject	financial-records	185.22.91.11	external	CRITICAL (90)	DENY
Sep 24, 2026, 03:51:48 AM	developer01	DeveloperRole	GetObject	credentials-vault	10.10.1.25	vpc-development	CRITICAL (100)	DENY
Sep 24, 2026, 03:51:48 AM	developer01	DeveloperRole	GetObject	credentials-vault	10.10.1.25	vpc-development	CRITICAL (100)	DENY
Sep 24, 2026, 03:51:48 AM	developer01	DeveloperRole	GetObject	credentials-vault	10.10.1.25	vpc-development	CRITICAL (100)	DENY
Sep 24, 2026, 03:51:48 AM	developer01	DeveloperRole	DeleteObject	credentials-vault	10.10.1.25	vpc-development	CRITICAL (100)	DENY
Sep 24, 2026, 03:51:48 AM	finance01	FinanceRole	GetObject	financial-records	203.0.113.25	external	CRITICAL (100)	DENY
Sep 24, 2026, 03:51:47 AM	developer01	DeveloperRole	GetObject	financial-records	185.22.91.11	external	CRITICAL (100)	DENY
Sep 24, 2026, 03:51:47 AM	contractor01	ContractorRole	GetObject	financial-records	185.22.91.11	external	CRITICAL (100)	DENY

Figure 14. Access Request audit log — every evaluated request with its user, action, resource, risk level, and final decision.

IAM, VPC Security, and Storage

The IAM, VPC Security, and Storage pages present the live reference catalog that every engine evaluates against — users and roles, network trust boundaries, and protected resources, respectively, shown in Figures 15–17.

USERNAME	DISPLAY NAME	ROLE	STATUS	CREATED
developer01	Developer One	DeveloperRole	Active	Sep 24, 2026, 03:51:25 AM
developer02	Developer Two	ReadOnlyRole	Active	Sep 24, 2026, 03:51:25 AM
admin01	Admin One	AdminRole	Active	Sep 24, 2026, 03:51:25 AM
finance01	Finance One	FinanceRole	Active	Sep 24, 2026, 03:51:25 AM
contractor01	Cloud Storage Contractor	ContractorRole	Active	Sep 24, 2026, 03:51:25 AM

Role	Permissions
DeveloperRole Application developers working on project data	GetObject:project-data, PutObject:project-data, ListBucket:project-data, GetObject:public-assets, ListBucket:public-assets
AdminRole Platform administrators; broad access for simulation purposes	...

Figure 15. IAM Management — the seeded users table and role/permission catalog evaluated by the IAM engine.

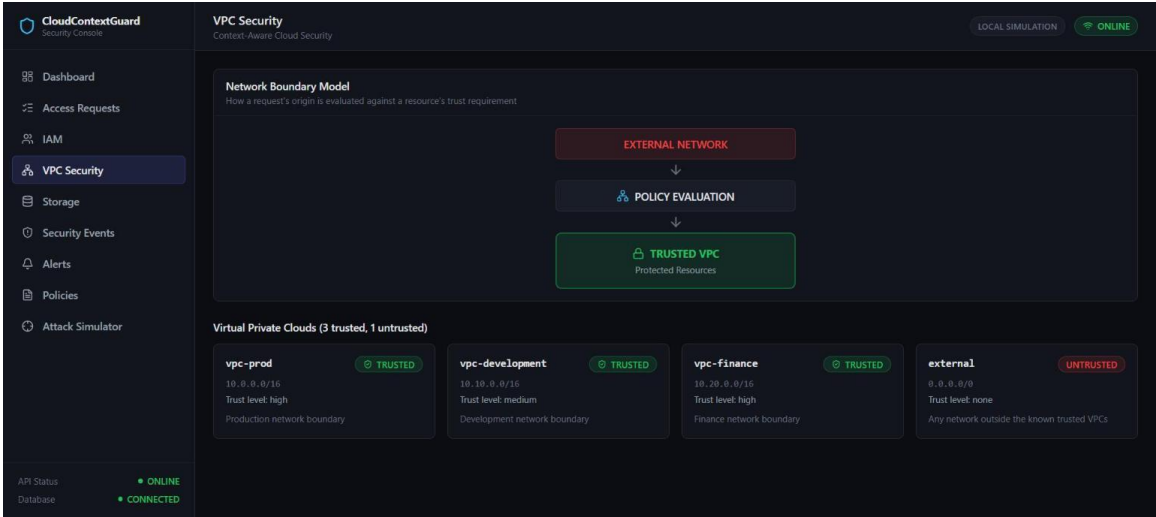


Figure 16. VPC Security — the network boundary model and the four simulated VPCs (three trusted, one untrusted external network).

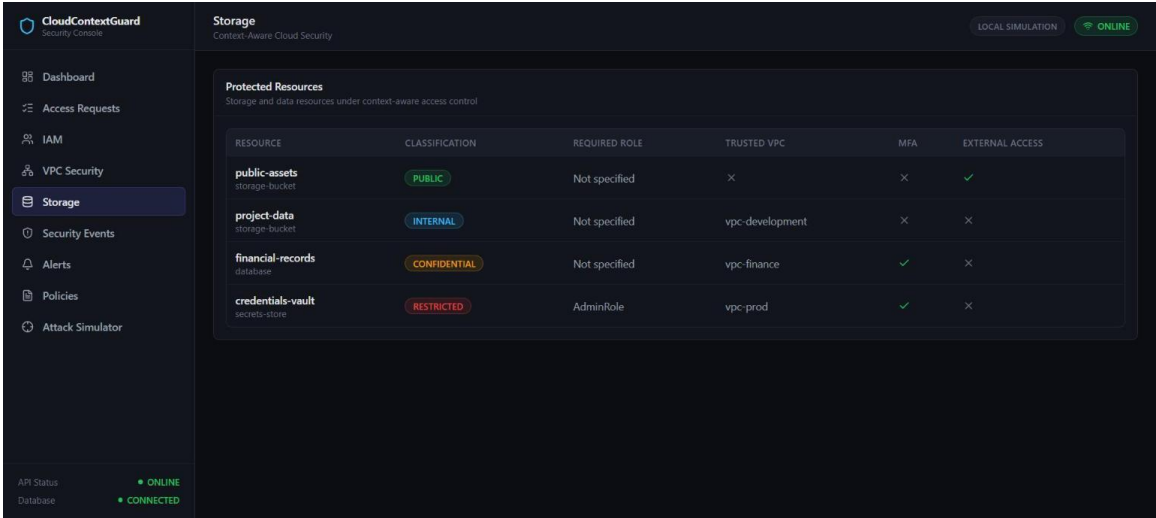


Figure 17. Storage — the four protected resources and their classification, trusted VPC, MFA, and external-access configuration.

Policies

The Policies page is the live editor described in Section 15, with summary tiles for total, enabled, MFA-protected, external-access, and high-risk policies, shown in Figure 18.

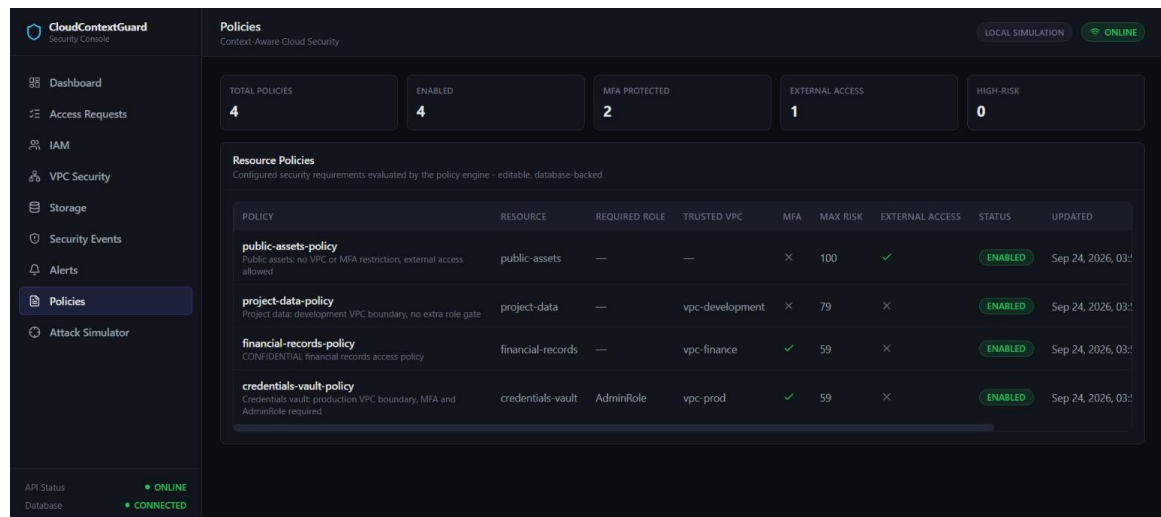


Figure 18. Policy Management — the four live, administrator-editable resource policies with their current configuration.

Attack Simulator

The Attack Simulator page presents six predefined, real scenarios, each of which submits a genuine request to the live API and renders the true result, shown in Figure 19.

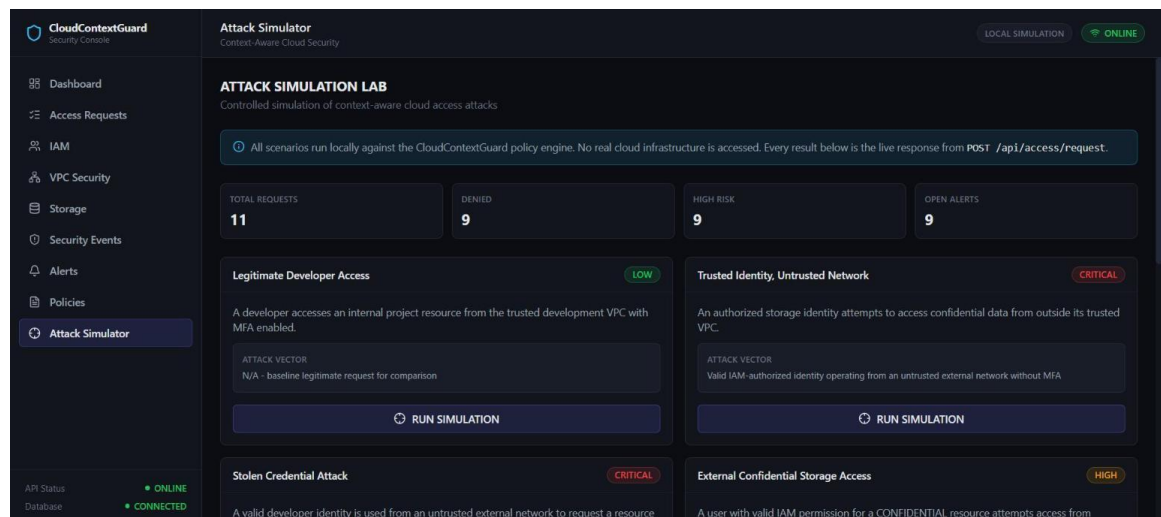


Figure 19. Attack Simulator landing page — six real, backend-verified attack scenarios, including the Topic 19 “Trusted Identity, Untrusted Network” scenario.

17. API Overview

The backend exposes a full REST API under `/api`, documented directly from the implementation in `API_DOCUMENTATION.md` and available as interactive documentation (Swagger UI) at `/api/docs` while the backend is running. Every list endpoint accepts an optional limit query parameter (default 50, range 1–500).

Table 10. Summary of API endpoints (source: `API_DOCUMENTATION.md`).

Method	Endpoint	Purpose
GET	<code>/api/health</code>	Report API and database health.
GET	<code>/api/dashboard</code>	Live, database-derived summary for the Dashboard page.
POST	<code>/api/access/request</code>	The single authorization entry point — evaluates a request through every engine and returns the full explainable result.
GET	<code>/api/access/requests</code>	Audit log of stored access requests, newest first.
GET	<code>/api/security/events</code>	The security event log (created on every DENY and every policy change).
GET	<code>/api/alerts</code>	Alerts raised for any HIGH or CRITICAL risk request, allowed or denied.
GET	<code>/api/users</code>	The catalog of seeded users.
GET	<code>/api/roles</code>	The catalog of roles.
GET	<code>/api/permissions</code>	The catalog of role-to-permission grants.
GET	<code>/api/resources</code>	The catalog of protected resources.
GET	<code>/api/vpcs</code>	The catalog of simulated network boundaries.
GET	<code>/api/policies</code>	The list of live, administrator-editable resource policies.
GET	<code>/api/policies/{id}</code>	A single policy by id.
PUT	<code>/api/policies/{id}</code>	The only way to change a policy; validates, applies, and audits the change.
POST	<code>/api/database/seed</code>	Idempotent, additive-only baseline catalog and demonstration-identity seeding.

The access-request and policy-update request bodies never accept a decision, risk score, id, or timestamp field from the client — these are either not present on the request schema at all, or are silently ignored and always recomputed or set by the server, per the mass-assignment resistance control described in Section 12.

18. Database Design

The system uses SQLite via SQLAlchemy 2.0, with foreign-key constraints enforced on every connection (`PRAGMA foreign_keys=ON`). The database file path is resolved dynamically at runtime (`backend/app/core/paths.py`) and is never hardcoded, which is what allows the entire project to be relocated to a different drive or folder without any source change. A lightweight schema-version mechanism guarantees that normal startup never drops a table or deletes data, and that an incompatible on-disk schema fails startup loudly instead of being silently recreated. Figure 20 shows the complete entity-relationship diagram for all ten tables in the schema.

CloudContextGuard — Database Entity Relationship Diagram

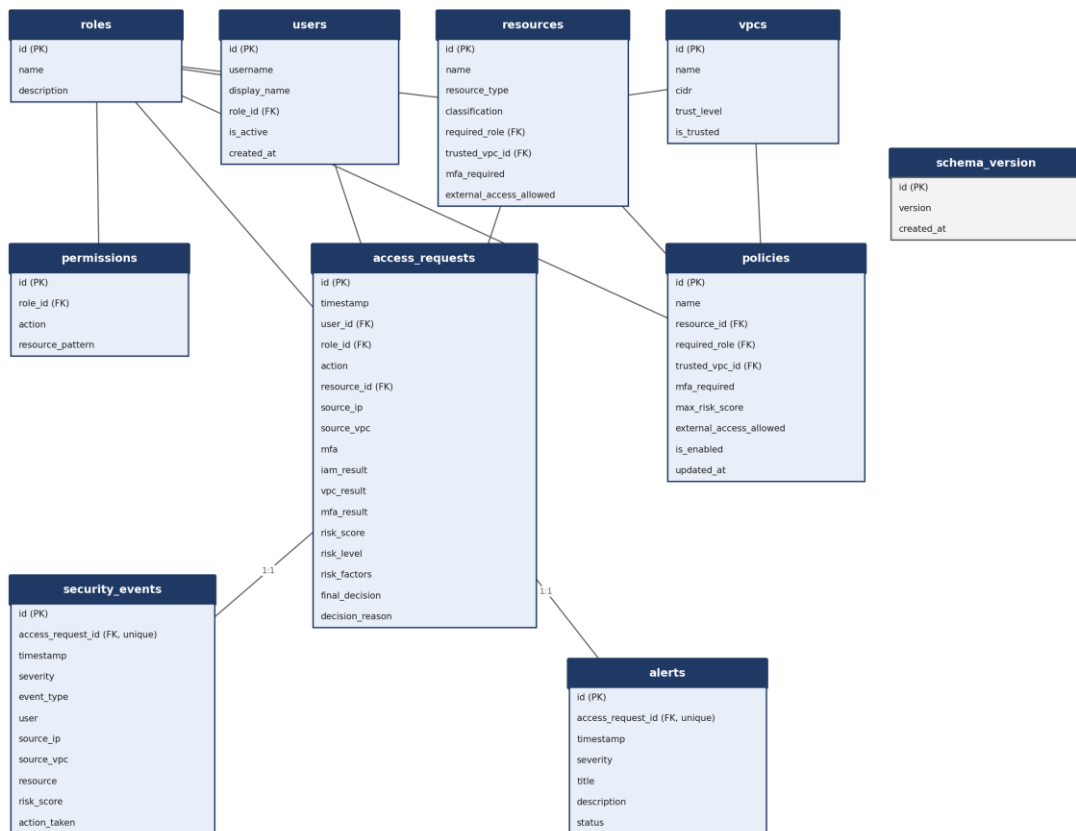


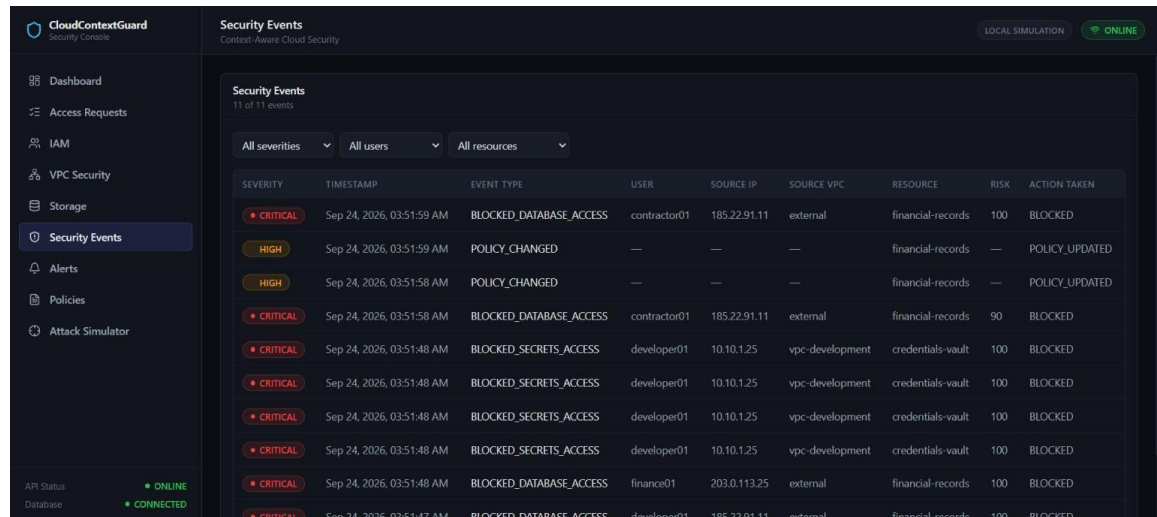
Figure 20. CloudContextGuard database entity-relationship diagram — all ten tables, their columns, and their foreign-key relationships.

Table 11. Database tables and their purpose (source: docs/architecture/PROJECT_STRUCTURE.md).

Table	Purpose
roles	Named roles (e.g. DeveloperRole, ContractorRole).
users	Simulated identities — username, display name, role, active flag.
permissions	Role → action / resource-pattern grants (IAM).
resources	Simulated protected resources — classification, baseline VPC/MFA/external-access defaults.
vpcs	Simulated network boundaries — CIDR, trust level, is_trusted.
policies	The live, administrator-editable authorization configuration per resource.
access_requests	Full audit record of every evaluated request and its outcome.
security_events	Denied-request and policy-change security log.
alerts	HIGH/CRITICAL-risk alerts, linked to their originating request.
schema_version	Single-row schema-compatibility stamp (the database-safety mechanism).

19. Security Events and Alerts

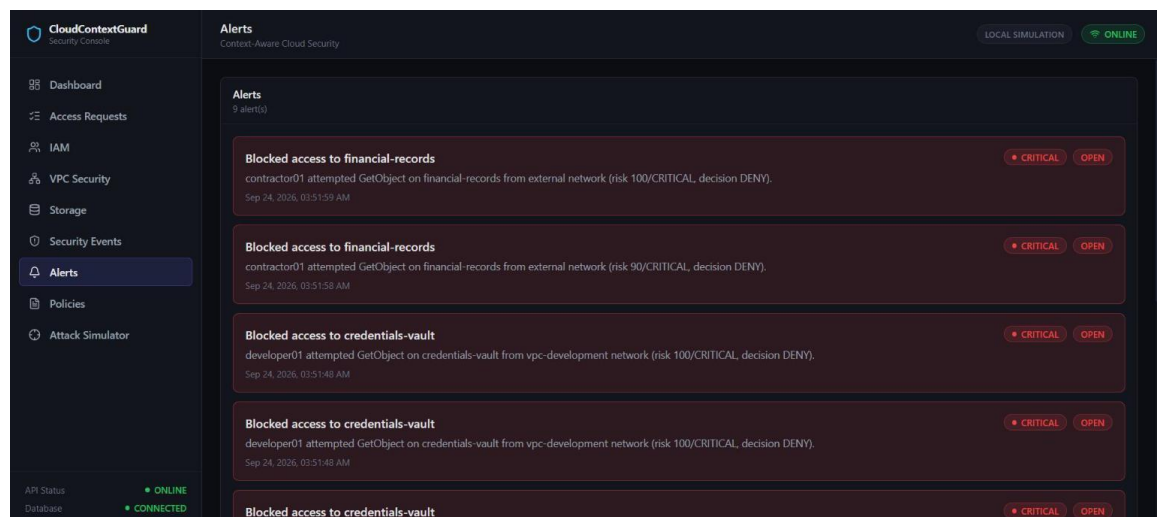
SecurityEvent rows are created in exactly two circumstances: every denied access request (never an allowed one), and every applied policy change. Fields captured include timestamp, severity, event_type, user, source_ip, source_vpc, resource, risk_score, a human-readable description, and action_taken; access_request_id is nullable and is null specifically for policy-change events. Figure 21 shows the live security event log, including the BLOCKED_DATABASE_ACCESS events generated by the demonstrations in Sections 14–15 and the POLICY_CHANGED events generated by the dynamic-policy experiment.



SEVERITY	TIMESTAMP	EVENT TYPE	USER	SOURCE IP	SOURCE VPC	RESOURCE	RISK	ACTION TAKEN
CRITICAL	Sep 24, 2026, 03:51:59 AM	BLOCKED_DATABASE_ACCESS	contractor01	185.22.91.11	external	financial-records	100	BLOCKED
HIGH	Sep 24, 2026, 03:51:59 AM	POLICY_CHANGED	—	—	—	financial-records	—	POLICY_UPDATED
HIGH	Sep 24, 2026, 03:51:58 AM	POLICY_CHANGED	—	—	—	financial-records	—	POLICY_UPDATED
CRITICAL	Sep 24, 2026, 03:51:58 AM	BLOCKED_DATABASE_ACCESS	contractor01	185.22.91.11	external	financial-records	90	BLOCKED
CRITICAL	Sep 24, 2026, 03:51:48 AM	BLOCKED_SECRETS_ACCESS	developer01	10.10.1.25	vpc-development	credentials-vault	100	BLOCKED
CRITICAL	Sep 24, 2026, 03:51:48 AM	BLOCKED_SECRETS_ACCESS	developer01	10.10.1.25	vpc-development	credentials-vault	100	BLOCKED
CRITICAL	Sep 24, 2026, 03:51:48 AM	BLOCKED_SECRETS_ACCESS	developer01	10.10.1.25	vpc-development	credentials-vault	100	BLOCKED
CRITICAL	Sep 24, 2026, 03:51:48 AM	BLOCKED_SECRETS_ACCESS	developer01	10.10.1.25	vpc-development	credentials-vault	100	BLOCKED
CRITICAL	Sep 24, 2026, 03:51:48 AM	BLOCKED_DATABASE_ACCESS	finance01	203.0.113.25	external	financial-records	100	BLOCKED
CRITICAL	Sep 24, 2026, 03:51:47 AM	BLOCKED_DATABASE_ACCESS	developer01	185.22.91.11	external	financial-records	100	BLOCKED

Figure 21. Security Events — the live audit log, including BLOCKED_DATABASE_ACCESS entries and two POLICY_CHANGED events from the dynamic-policy experiment.

Alert rows are created for any request — allowed or denied — assessed as HIGH or CRITICAL risk, linked to the originating request through a unique access_request_id, which guarantees at most one alert per request. Severity mirrors the risk level, and status starts OPEN. Figure 22 shows the live alerts list for the current demonstration dataset.



Alert Title	Description	Timestamp	Severity	Status
Blocked access to financial-records	contractor01 attempted GetObject on financial-records from external network (risk 100/CRITICAL, decision DENY).	Sep 24, 2026, 03:51:59 AM	CRITICAL	OPEN
Blocked access to financial-records	contractor01 attempted GetObject on financial-records from external network (risk 90/CRITICAL, decision DENY).	Sep 24, 2026, 03:51:58 AM	CRITICAL	OPEN
Blocked access to credentials-vault	developer01 attempted GetObject on credentials-vault from vpc-development network (risk 100/CRITICAL, decision DENY).	Sep 24, 2026, 03:51:48 AM	CRITICAL	OPEN
Blocked access to credentials-vault	developer01 attempted GetObject on credentials-vault from vpc-development network (risk 100/CRITICAL, decision DENY).	Sep 24, 2026, 03:51:48 AM	CRITICAL	OPEN
Blocked access to credentials-vault	developer01 attempted GetObject on credentials-vault from vpc-development network (risk 100/CRITICAL, decision DENY).	Sep 24, 2026, 03:51:47 AM	CRITICAL	OPEN

Figure 22. Alerts — every HIGH/CRITICAL-risk request in the current demonstration dataset, each linked to its originating access request.

20. Testing and Validation

Validation followed two complementary methods throughout the project: automated testing — 82 pytest tests exercising the real API (via FastAPI's TestClient) and, for the risk and decision engines, direct unit-level calls to the pure engine functions, all against the project's real database; and live demonstration — the same scenarios re-executed against a running uvicorn server and a running React frontend, with results independently cross-checked against the automated test results and the raw API JSON. All figures in this section are directly reproducible by running `pytest` from `backend/`, `python tests/verify_portability.py` from the project root, and `npm run build` from `frontend/`.

Table 12. *Automated backend test suite (source: docs/research/RESULTS.md; re-verified for this documentation).*

Suite	File	Tests
Health / liveness	backend/tests/test_health.py	5
Access control (IAM/VPC/MFA/risk/policy, contractor01)	backend/tests/test_access_control.py	17
Policy management (CRUD, validation, dynamic policy)	backend/tests/test_policy_management.py	10
Database safety (schema-version, persistence, reset)	backend/tests/test_database_safety.py	9
Security validation (Step 7 comprehensive pass)	backend/tests/test_security_validation.py	41
Total		82 passed, 0 failed

Portability was independently re-verified using `tests/verify_portability.py`: 17 of 17 checks passed, including project-root detection independent of the working directory, a simulated moved project folder, every named project path, and a scan of every source and configuration file for hardcoded, machine-specific paths. The frontend production build (`npm run build`, Vite) completed successfully with no errors.

Note: All three results — 82/82 backend tests, 17/17 portability checks, and the frontend build — were independently re-executed against the project's real source code as part of preparing this documentation, rather than copied from a prior record.

21. Results

The complete, verified results of this project are summarized below and presented in full detail in Sections 14, 15, and 20.

Table 13. Consolidated results summary.

Metric	Result
Backend automated tests	82 passed, 0 failed
Portability checks	17 of 17 passed
Frontend production build	Passed, no errors
Primary contextual-denial demonstration (contractor01, external, no MFA)	IAM PASS, VPC FAIL, MFA FAIL, risk 100/CRITICAL, DENY
Dynamic-policy demonstration, after live policy edit	IAM PASS, VPC PASS, risk 40/MEDIUM, ALLOW
Dynamic-policy demonstration, after policy restored	VPC FAIL, DENY (matches original baseline)

The project's central finding, reproduced independently by both the automated test suite and live demonstration against the running application, is direct: a genuinely valid IAM permission, exercised from an untrusted network without MFA, is still denied. The dynamic-policy experiment additionally shows that this behavior is not fixed at deploy time — an administrator can reconfigure which network is trusted for a given resource at runtime, and the very next request is evaluated against the new configuration, with the system fully returnable to its original, secure configuration by the same mechanism.

Note: These results describe the behavior of a local, controlled simulation only. No finding here should be read as a claim about the security properties of any real AWS deployment or any production system.

22. Limitations

- Local simulation only; no connection to a real cloud provider.
- No real AWS integration and no real cloud credentials are used or supported anywhere in this project.
- Risk scoring is rule-based (fixed, named point values), not machine-learning-based.
- No production-grade rate limiting is implemented.
- SQLite is used rather than a production distributed database.
- Schema-change safety uses a lightweight, manually-maintained schema-version number, not a full migration framework such as Alembic.
- No production identity provider (OAuth / OIDC / SAML) is integrated; identities are rows in the local database with no authentication of their own.
- No real cloud telemetry (e.g. AWS CloudTrail) is ingested; all audit data originates from this application's own simulated requests.

23. Future Work

- AWS IAM integration map simulated roles/permissions onto real IAM policy evaluation.
- AWS VPC endpoint context real network-origin verification via VPC endpoints or PrivateLink.
- S3 integration apply the same context-aware model to real S3 bucket policies.
- CloudTrail ingestion replace simulated audit data with real AWS event data.
- Real-time event streaming push security events and alerts to the frontend rather than polling.
- ML-based anomaly detection, as a complement to not a replacement for the current rule-based risk engine.
- Production-grade rate limiting.
- A distributed database for production-scale audit volume.
- Identity-provider integration (OAuth / OIDC / SAML).
- Policy-as-code and Terraform / infrastructure-as-code integration for managing policies outside the application database.

24. Conclusion

CloudContextGuard demonstrates, with a fully working and independently testable system, that context-aware cloud authorization identity, network, MFA, policy, and risk evaluated together, rather than identity permission alone is implementable, explainable, and dynamically configurable, without sacrificing server-side authority. Its central finding, reproduced by both automated tests and live demonstration, is direct: a genuinely valid IAM permission, exercised from an untrusted network without MFA, is still denied.

CloudContextGuard operationalizes a core zero-trust principle “**never trust, always verify**” applied specifically to cloud storage access, as a working, testable system rather than a slide. It gives a concrete, reproducible answer to the question “what happens to a request with a stolen but genuinely valid credential, from an unexpected network, without MFA?” and demonstrates that dynamic, database-driven policy is a viable mechanism for enforcing contextual IAM restrictions without redeploying code.

25. References

- [1] CloudContextGuard project repository. <https://github.com/priyaansh21/CloudContextGuard>
- [2] CloudContextGuard, README.md — project overview, architecture, and usage.
- [3] CloudContextGuard, PROJECT_DOCUMENTATION.md — full project report.
- [4] CloudContextGuard, THREAT_MODEL.md — the seven simulated threats (T1–T7).
- [5] CloudContextGuard, API_DOCUMENTATION.md — complete REST API reference.
- [6] CloudContextGuard, docs/research/SECURITY_DECISION_MODEL.md — risk formula and decision thresholds.
- [7] CloudContextGuard, docs/research/DYNAMIC_POLICY_EVALUATION.md — the dynamic-policy experiment.
- [8] CloudContextGuard, docs/research/METHODOLOGY.md — research methodology and validation approach.
- [9] CloudContextGuard, docs/research/RESULTS.md — verified test, portability, and demonstration results.
- [10] CloudContextGuard, docs/architecture/PROJECT_STRUCTURE.md — full annotated directory tree and database schema.
- [11] Rose, S., Borchert, O., Mitchell, S., and Connelly, S. Zero Trust Architecture. NIST Special Publication 800-207, National Institute of Standards and Technology, 2020.
- [12] OWASP Foundation. OWASP Top 10:2021 — The Ten Most Critical Web Application Security Risks.
- [13] FastAPI — official documentation. <https://fastapi.tiangolo.com/>
- [14] SQLAlchemy 2.0 — official documentation. <https://docs.sqlalchemy.org/>
- [15] React — official documentation. <https://react.dev/>